THE GRID

The Global Resilient Internet Datalink

n3o

Abstract

THE GRID defines a peer-to-peer encrypted storage network in which participating nodes—called **ZODES**—form a decentralized mesh of program-scoped, append-only log stores. All data is encrypted client-side before upload; ZODES store and serve only opaque ciphertext, ensuring that node operators never observe plaintext content, authorship, or access patterns beyond coarse timing and payload-size metadata. The protocol has no consensus layer, no tokens, and no global state.

Cryptographic identity is rooted in a 256-bit *NeuralKey* from which all signing and encryption keys are deterministically derived via HKDF-SHA256. Signatures use a post-quantum hybrid construction pairing Ed25519 with ML-DSA-65, and key agreement combines X25519 with ML-KEM-768, so that an attacker must break both the classical and post-quantum primitives. Sector encryption supports two algorithms—XChaCha20-Poly1305 for general use and a ZK-friendly Poseidon sponge over the BN254 scalar field—chosen per program. In Poseidon mode, clients may attach Groth16 *shape proofs* that let ZODES verify ciphertext integrity and schema conformance without seeing any plaintext.

Networking is built on libp2p with QUIC as the primary transport. Data propagation uses GossipSub topics keyed by program identifier; peer discovery relies on Kademlia DHT with periodic random walks. The wire format is deterministic CBOR (RFC 8949), and all identifiers—program IDs, sector IDs, content IDs—are derived via SHA-256 hashes of canonical CBOR, ensuring cross-implementation reproducibility. Above the storage substrate, a Services layer provides a trait-based framework for stateless servers that run on ZODES, use Programs for persistent state via a key-value abstraction over sector logs, and expose HTTP endpoints. Services are pluggable, discoverable, and support both native (compiled Rust) and sandboxed WASM runtimes via the WebAssembly Component Model. This document specifies the serialization, cryptography, storage model, wire protocol, gossip replication, proof system, services framework, and standard programs (ZID identity and Interlink messaging) in sufficient detail for independent, interoperable implementations.

1 Introduction

The Grid is a peer-to-peer encrypted storage protocol. Participating nodes—called **ZODES**—form a network of program-scoped, append-only log stores. Clients encrypt all data locally before upload; ZODES store and serve only opaque ciphertext. The protocol has no consensus layer, no tokens, and no global state.

Existing decentralized storage systems typically require participants to reach consensus on a shared ledger, introduce protocol-native tokens to incentivize storage providers, or maintain globally replicated state that every node must process. These design choices impose coordination overhead, economic complexity, and scalability ceilings that are unnecessary when the primary goal is simply to store and retrieve encrypted blobs. The Grid sidesteps these concerns entirely: a ZODE is a stateless relay that accepts ciphertext, appends it to a local log, and optionally gossips it to peers. There is no ordering protocol beyond append indexing, no fee market, and no finality mechanism. The result is a minimal, high-throughput substrate on which higher-level applications—identity registries, messaging channels, document stores—can be composed as *programs* without inheriting the weight of a full blockchain stack. A Services layer built atop this substrate provides a standard framework for stateless servers that use programs for persistent state and expose HTTP endpoints, supporting both native and sandboxed WASM runtimes.

Cryptographic privacy is a first-class invariant rather than an optional feature. Every entry is encrypted client-side with either XChaCha20-Poly1305 or a ZK-friendly Poseidon sponge, and signatures are embedded inside the ciphertext so that ZODES can never observe authorship, content structure, or access patterns beyond coarse timing and payload size. Post-quantum

resistance is likewise non-negotiable: all signing and key-agreement operations use hybrid constructions that pair classical algorithms (Ed25519, X25519) with their post-quantum counterparts (ML-DSA-65, ML-KEM-768), ensuring that an attacker must break both families simultaneously. This document defines the Grid protocol in implementation-neutral terms. Any conforming implementation—regardless of programming language—that follows the wire formats, cryptographic constructions, and behavioral rules described here can interoperate with any other conforming implementation.

2 Terminology

Term	Definition
ZODE	A storage node that participates in the Grid network. Identified by a libp2p PeerId, displayed with a `Zx` prefix. The prefix is display-only; wire and storage formats use the raw PeerId.
Client	Any participant that connects to ZODES to store or retrieve data. A client does not serve requests.
Program	A named, versioned application scope. All storage, subscriptions, and topic routing are scoped by program.
ProgramId	A 32-byte identifier derived as `SHA-256(CBOR(ProgramDescriptor))`.
Sector	An append-only log of encrypted entries identified by a `(ProgramId, SectorId)` pair.
SectorId	An opaque 32-byte identifier. In the sector protocol, sector IDs MUST be exactly 32 bytes.
Entry	A single encrypted blob appended to a sector log. Entries are indexed 0, 1, 2, …
NeuralKey	A 256-bit root secret from which all identity and machine keys are deterministically derived.
SectorKey	A random 256-bit symmetric key used to encrypt sector entries (§6).
Topic	A GossipSub topic string of the form `prog/<program_id_hex>` (programs) or `svc/<service_id_hex>` (services).
Service	An active, stateless process that runs on a ZODE, uses Programs for persistent state, and exposes HTTP endpoints. Contrast with Programs, which are passive descriptors.
ServiceId	A 32-byte identifier derived as `SHA-256(CBOR(ServiceDescriptor))`.
ServiceDescriptor	A CBOR map declaring a service's name, version, required programs, and owned programs.
ProgramStore	A key-value abstraction over sector append-logs provided to services. Keys map to SectorIds via `SHA-256(key)`; values are sector log entries.

3 Serialization

All protocol messages, program descriptors, and gossip payloads use **CBOR** (RFC 8949) with deterministic encoding (RFC 8949 §4.2.1).

Implementations MUST produce identical byte sequences for identical logical values. Field ordering within CBOR maps MUST be consistent. The canonical CBOR bytes are the input to all hash operations (ProgramId derivation, CID computation, etc.). Binary fields (payloads, keys, ciphertexts) are encoded as CBOR byte strings.

4 Identifiers

4.1 ProgramId

A 32-byte value:

```
ProgramId = SHA-256(CBOR(ProgramDescriptor))
```

Displayed as 64 lowercase hex characters.

4.2 ProgramDescriptor

A CBOR map with at minimum:

Field	Type	Description
`name`	text string	Short program name
`version`	unsigned integer (u32)	Program version number
`proof_required`	boolean	Whether Valid-Sector proofs are required
`proof_system`	optional ProofSystem	`None` → XChaCha20-Poly1305; `Groth16` → Poseidon sponge + shape proofs (§6, §13)

Program-specific descriptors MAY add additional fields. Two implementations that serialize the same descriptor fields to CBOR MUST produce the same ProgramId.

4.3 SectorId

A variable-length byte string. In the sector protocol, sector IDs MUST be exactly 32 bytes. ZODES MUST reject requests with non-32-byte sector IDs. Displayed as lowercase hex.

4.4 Cid (Content Identifier)

```
Cid = SHA-256(ciphertext)
```

A 32-byte content-addressed identifier derived from the stored ciphertext. Displayed as 64 lowercase hex characters.

4.5 ServiceId

A 32-byte value:

```
ServiceId = SHA-256(CBOR(ServiceDescriptor))
```

Displayed as 64 lowercase hex characters. Mirrors the ProgramId derivation pattern: a canonical CBOR-serialized descriptor hashed to produce a deterministic, content-addressed identifier.

4.6 ZODE ID

A libp2p PeerId. On the wire and in storage, the raw PeerId bytes are used. For human display, the canonical format is `Zx<PeerId_string>`. The `Zx` prefix MUST NOT appear in wire formats.

5 Cryptographic Primitives

5.1 Key Hierarchy

All keys derive from a **NeuralKey**—a 256-bit secret generated by CSPRNG.

```
NeuralKey (256-bit CSPRNG)
|
+-- IdentitySigningKey (Ed25519 + ML-DSA-65)
|   Derived with: identity_id (16 bytes)
|
+-- MachineKeyPair (per device, per epoch)
    Derived with: identity_id, machine_id, epoch
    Contains:
    +-- Ed25519      (classical signing)
    +-- X25519       (classical key agreement)
    +-- ML-DSA-65    (post-quantum signing)
    +-- ML-KEM-768   (post-quantum encapsulation)
```

All four key types are always present in every MachineKeyPair. There is no classical-only mode.

5.2 Key Derivation

All derivation uses **HKDF-SHA256** with `salt = None` unless otherwise specified.

Identity Signing Key

Two separate derivations from the NeuralKey:

Component	HKDF info
Ed25519 seed	`"cypher:id:identity:v1" || identity_id`
ML-DSA-65 seed	`"cypher:id:identity:pq-sign:v1" || identity_id`

Machine Key Pair

Two-level derivation. **Step 1**—Machine seed:

```
machine_seed = HKDF-SHA256(
    ikm  = NeuralKey,
    info = "cypher:shared:machine:v1"
           || identity_id || machine_id
           || epoch_be_bytes
)
```

Step 2—Individual key seeds from machine seed:

Component	HKDF info
Ed25519 signing	`"cypher:shared:machine:sign:v1" || machine_id`
X25519 encryption	`"cypher:shared:machine:encrypt:v1" || machine_id`
ML-DSA-65 signing	`"cypher:shared:machine:pq-sign:v1" || machine_id`
ML-KEM-768 encap	`"cypher:shared:machine:pq-encrypt:v1" || machine_id`

ML-KEM-768 keys are generated deterministically: the PQ encrypt seed is expanded via HKDF into `d` (`"mlkem768:d"`) and `z` (`"mlkem768:z"`) parameters, which are passed to ML-KEM-768 deterministic key generation.

5.3 Machine Key Capabilities

Machine keys carry a bitflag:

Bit	Name	Value
0	SIGN	0x01
1	ENCRYPT	0x02
2	STORE	0x04
3	FETCH	0x08

Capabilities are metadata; enforcement is at the application/policy layer.

5.4 Hybrid Signatures

A **HybridSignature** always contains both components:

Component	Size	Description
Ed25519	64 bytes	Classical signature
ML-DSA-65	3,309 bytes	Post-quantum signature

Binary format: `ed25519_bytes (64) || ml_dsa_bytes (3309)`

Both components sign the same message. **Both MUST verify** for the signature to be considered valid.

5.5 Hybrid Key Encapsulation

Key agreement combines X25519 and ML-KEM-768:

```
x25519_ss = X25519(sender_secret,
                   recipient_public)
(mlkem_ct, mlkem_ss) =
    ML-KEM-768.Encapsulate(recipient_pk)

shared_secret = HKDF-SHA256(
    ikm  = x25519_ss || mlkem_ss,
    salt = None,
    info = "zid:encap:v1"
) -> 32 bytes
```

The output is a `SharedSecret` (32 bytes) and an `EncapBundle`:

Field	Size	Description
`x25519_public`	32 bytes	Sender's static X25519 public key
`mlkem_ciphertext`	1,088 bytes	ML-KEM-768 ciphertext

An attacker must break **both** X25519 and ML-KEM-768 to recover the shared secret.

5.6 DID Encoding

Ed25519 public keys are encoded as `did:key` identifiers: `did:key:z` + `base58btc(0xed01 || ed25519_public_key_bytes)`. The multicodec prefix `0xed01` identifies Ed25519 public keys.

5.7 Shamir Secret Sharing

The NeuralKey MAY be split into Shamir shares for backup and recovery. **Split**: `split(secret[32], total, threshold, rng)`. **Combine**: `combine(shares[≥ threshold])`. Each share is serialized as hex: `hex(index_byte || share_data)`.

Identity generation, signing, and machine key derivation can all operate by **ephemerally reconstructing** the NeuralKey from shares, performing the operation, and immediately zeroizing the secret.

6 Sector Encryption

Sector encryption supports two algorithms. The choice is per-program via `proof_system` in the ProgramDescriptor (§4.2): `None` uses XChaCha20-Poly1305; `Groth16` uses Poseidon sponge (required for shape proofs, §13).

6.1 SectorKey

A SectorKey is a random 256-bit symmetric key generated via CSPRNG. Key wrapping (§6.5) is unchanged regardless of encryption algorithm.

6.2 XChaCha20-Poly1305

```
sealed = nonce (24 bytes) || ciphertext || tag (16 bytes)
```

Nonce: 192-bit, randomly generated per encryption. **AAD:** `program_id (32) || sector_id (32)`, binding ciphertext to its program and sector.

6.3 Poseidon Sponge

Parameters: BN254 scalar field, rate=2, capacity=1.

```
sealed = nonce (32 bytes)
    || ciphertext_elements (32 bytes each)
    || tag (32 bytes)
```

- **Nonce:** 256-bit, randomly generated per encryption.
- **AAD:** `program_id || sector_id` absorbed into the sponge before the plaintext.
- **Field element packing:** Plaintext is split into chunks of 30 data bytes. Each chunk is packed into a 32-byte field element as `[1-byte length][up to 30 bytes data][zero-pad]`.
- **Duplex mode:** The sponge absorbs key, nonce, and AAD elements. For each rate-sized chunk it squeezes keystream, adds to plaintext to produce ciphertext, then absorbs plaintext.
- **Tag:** Final 32-byte squeeze authenticates the entire encryption.

6.4 Padding

Before encryption, content MUST be padded to fixed-size buckets to resist payload-size analysis. Buckets grow in 2× progression:

Content size (incl. 4-byte prefix)	Padded to
0 – 256 B	256 B
257 – 512 B	512 B
513 – 1,024 B	1 KB
1,025 – 2,048 B	2 KB
2,049 – 4,096 B	4 KB
4,097 – 8,192 B	8 KB
8,193 – 16,384 B	16 KB
16,385 – 32,768 B	32 KB
32,769 – 65,536 B	64 KB
65,537 – 131,072 B	128 KB
131,073 – 262,144 B	256 KB
> 262,144 B	Next 256 KB multiple

Padding format:

```
padded = content_length (4 bytes, LE)
    || content
    || 0x00 * (bucket_size - 4 - content_length)
```

6.5 Key Wrapping

Step 1—Hybrid key agreement: between sender's MachineKeyPair and the recipient's MachinePublicKey using §5.5, producing a `SharedSecret` and `EncapBundle`.

Step 2—Context-bound wrapping:

```
wrap_key = HKDF-SHA256(
  ikm  = SharedSecret,
  salt = None,
  info = "grid:sector-key-wrap:v1"
       || program_id || sector_id
)

wrapped_sector_key = XChaCha20-Poly1305(
  key       = wrap_key,
  nonce     = random 192-bit,
  plaintext = sector_key_bytes
)
```

The result is a **KeyEnvelopeEntry**:

Field	Type	Description
`recipient_did`	text	`did:key` of recipient
`sender_x25519_public`	bytes	Sender's X25519 public key (32 B)
`mlkem_ciphertext`	bytes	ML-KEM-768 ciphertext (1,088 B)
`wrapped_key`	bytes	`nonce(24) || enc_key(32) || tag(16)` = 72 B

6.6 Sector ID Derivation

For metadata-private storage, sector IDs are derived client-side via two-step HKDF:

```
derivation_key = HKDF-SHA256(
    ikm=shared_secret,
    salt="grid:sector:v1",
    info="grid:sector:derive-key:v1")

sector_id = HKDF-SHA256(
    ikm=derivation_key,
    salt="grid:sector:v1",
    info=<application-defined string>)
```

The intermediate derivation key MUST be zeroized after use. Info string convention: `"grid:{program}:{purpose}:{...fields}"`.

Properties: Deterministic (same secret + info → same sector ID), Unlinkable (different info strings → unrelated IDs), Collision-resistant (32-byte HKDF output).

7 Message Signing

Message signing provides authenticity and integrity for entries within the encrypted blob. Signatures are PQ-hybrid (Ed25519 + ML-DSA-65) and are produced before encryption, verified after decryption.

7.1 Placement

Signatures live **inside** the encrypted payload. The ZODE never sees plaintext; signature verification is end-to-end.

7.2 Signable Bytes

```
signable_bytes = canonical_cbor(
    all_fields_except_signature)
```

Fields MUST be serialized in deterministic CBOR order.

7.3 HybridSignature Format

`Ed25519 (64) || ML-DSA-65 (3309)` = 3,373 bytes total. Both components sign the same `signable_bytes`. **Both MUST verify.**

7.4 Signing and Verification Flow

1. **Signing:** Client computes `signable_bytes`, signs with both algorithms, appends signature, then encrypts.
2. **Verification:** Recipient decrypts, recomputes `signable_bytes`, verifies both Ed25519 and ML-DSA-65.

7.5 Ed25519-Only Fallback (v1)

For v1 compatibility, programs MAY support Ed25519-only signatures (64 bytes).

8 ZODE Storage Model

8.1 Append-Only Logs

Each sector is an ordered sequence of entries indexed starting at 0. Sectors are identified by `(ProgramId, SectorId)`.

8.2 Key Layout

```
key = program_id (32 B) || sector_id (32 B)
  || index (8 B, big-endian)
```

Total key size: 72 bytes.

8.3 Operations

Operation	Behavior
append	Reverse-seek to find max index, write at `max + 1`
insert_at	Write at specific index if unoccupied (idempotent)
read_log	Forward-iterate from `from_index`, return up to `max_entries`
log_length	Reverse-seek to find max index + 1 (0 if empty)

8.4 Policy Enforcement

Policy	Description
Program allowlist	Only serve programs in effective topic set. Reject: `PolicyReject`.
Sector filter	Optionally restrict to explicit sector ID set.
Entry size limit	Max 256 KB per entry. Reject: `InvalidPayload`.
Batch limits	Max 64 entries, 4 MB total. Reject: `BatchTooLarge`.
Shape proof	Verify `ShapeProof` when `proof_system = Groth16`. Reject: `ProofInvalid`.
Per-program quota	Optional max bytes per program.

9 Topic Naming

9.1 Program Topics

```
topic = "prog/" + hex(ProgramId)
```

Where `hex(ProgramId)` is the 64-char lowercase hex encoding of the 32-byte ProgramId. ZODES subscribe to one or more topics and only accept requests for subscribed programs.

9.2 Service Topics

```
topic = "svc/" + hex(ServiceId)
```

Services participate in GossipSub under `svc/`-prefixed topics. When a ZODE starts a service, it subscribes to that service's topic. This enables peer discovery of which ZODES serve a given service.

10 Transport Layer

10.1 Connection

The Grid uses **libp2p** with the following transports:

- **QUIC** (`/quic-v1`): Primary transport.
- **TCP + Noise + Yamux**: Fallback transport.

Default listen address: `/ip4/0.0.0.0/udp/3690/quic-v1`.

10.2 Protocols

Protocol String	Type	Purpose
`/grid/sector/1.0.0`	Request-response	Client ↔ ZODE sector ops
`/grid/kad/1.0.0`	Kademlia DHT	Peer discovery
GossipSub	Pub/sub	Data propagation
HTTP (`/services/`)	Request-response	Service endpoints (§15)

10.3 GossipSub Configuration

Message authentication: signed (libp2p keypair). Heartbeat interval: 10 s. Validation mode: permissive. Message ID: hash of message data (content-based dedup).

10.4 Peer Discovery

ZODES accept bootstrap peer multiaddrs in configuration. On startup, the ZODE dials each bootstrap peer. When Kademlia is enabled, the ZODE seeds the routing table, triggers initial `bootstrap()`, and periodically performs random walk queries (default: 30 s interval).

Parameter	Default
Protocol name	`/grid/kad/1.0.0`
Query timeout	60 s
Mode	Server (ZODES) / Client (SDK)
Random walk interval	30 s
Max concurrent dials	8

11 Sector Protocol — Wire Format

The sector protocol is the primary client-to-ZODE interface, operating over `/grid/sector/1.0.0`.

11.1 Request Envelope

```
SectorRequest = Append(SectorAppendRequest)
  | ReadLog(SectorReadLogRequest)
  | LogLength(SectorLogLengthRequest)
  | BatchAppend(SectorBatchAppendRequest)
  | BatchLogLength(SectorBatchLogLengthRequest)
```

11.2 Response Envelope

Response variants correspond 1:1 to request variants.

11.3 Append

Request:

Field	Type	Description
`program_id`	bytes(32)	Target program
`sector_id`	bytes(32)	Target sector
`entry`	byte string	Encrypted payload
`shape_proof`	optional	ShapeProof (§13)

Response: `ok` (boolean), `index` (optional uint64), `error_code` (optional).

11.4 ReadLog

Request: `program_id`, `sector_id`, `from_index` (uint64), `max_entries` (uint32, capped at 64). **Response:** `entries` (array of byte strings), `error_code` (optional). Empty array if sector does not exist.

11.5 LogLength

Request: `program_id`, `sector_id`. **Response:** `length` (uint64), `error_code` (optional).

11.6 BatchAppend

Request: `program_id`, `entries` (array of `BatchAppendEntry`, up to 64). Each entry: `sector_id`, `entry`, optional `shape_proof`. **Response:** `results` (array of `AppendResult`: `ok`, `index`, `error_code`).

11.7 BatchLogLength

Request: `program_id`, `sector_ids` (array of bytes(32), up to 64). **Response:** `results` (array of `LogLengthResult`: `length`, `error_code`).

11.8 Batch Limits

Maximum **64 entries** and **4 MB total payload** per batch. ZODES MUST reject with `BatchTooLarge`.

11.9 Error Codes

Code	Meaning
`StorageFull`	Storage capacity exceeded
`ProofInvalid`	Shape proof verification failed
`PolicyReject`	Policy violation
`NotFound`	Data not present
`InvalidPayload`	Malformed request or oversized entry
`ProgramMismatch`	Wrong program for subscription
`SlotOccupied`	Reserved (write-once)
`BatchTooLarge`	Batch limits exceeded
`ConditionFailed`	Reserved (conditional write)

Error codes are serialized as CBOR text strings.

12 Gossip Replication

When a ZODE accepts an `Append` request, it publishes a `GossipSectorAppend` message to the program's GossipSub topic. Other subscribed ZODES store the entry automatically.

12.1 GossipSectorAppend

Field	Type	Description
`program_id`	bytes(32)	Program
`sector_id`	bytes(32)	Sector
`index`	uint64	Assigned log index
`payload`	byte string	Encrypted entry
`shape_proof`	optional	ShapeProof (§13)

12.2 Receiving ZODE Behavior

1. **Program check:** Discard if not serving `program_id`.

2. **Sector filter:** Check allowlist if configured.
3. **Entry size check:** Discard if oversized.
4. **Shape proof:** Verify if required; discard on failure.
5. **Idempotent insert:** Store at given index if unoccupied; silently ignore byte-identical duplicates; on conflict, append at next available index.
6. **No re-gossip:** GossipSub mesh handles fan-out.

13 Shape Proofs

Shape proofs allow clients to prove that an encrypted blob conforms to a declared schema without revealing plaintext. Programs with `proof_system = Groth16` use Poseidon sponge encryption (§6.3) and MAY attach a shape proof to each entry.

13.1 ProofSystem Enum

Variant	Encryption	Shape proofs
`None`	XChaCha20-Poly1305	Not supported
`Groth16`	Poseidon sponge	Supported

13.2 FieldSchema

Message structure is described by a **FieldSchema**: a sequence of `FieldDef` (named field + `CborType`).

```
schema_hash = SHA-256(canonical_cbor(schema))
```

13.3 ShapeProof Wire Format

Field	Type	Description
`proof_system`	ProofSystem	`Groth16`
`ciphertext_hash`	bytes(32)	Poseidon(C)
`proof_bytes`	byte string	Groth16 proof
`schema_hash`	bytes(32)	SHA-256 of schema
`size_bucket`	uint32	Circuit bucket size

13.4 Universal Circuit

The shape-proof circuit proves three predicates:

1. **shape(B)**—Plaintext B conforms to the FieldSchema.
2. **Poseidon_encrypt(B, K, N, AAD) = C**—Ciphertext C is the correct encryption.
3. **Poseidon(C) = ciphertext_hash**—Public hash matches.

13.5 Strong Binding

The ZODE computes `Poseidon(received_ciphertext)` and MUST verify it equals the attested `ciphertext_hash`.

13.6 Message-Size Buckets

Bucket	Max size
1 KB	1,024 bytes
4 KB	4,096 bytes
16 KB	16,384 bytes
64 KB	65,536 bytes

Version 1: Only 1 KB and 4 KB buckets are supported. One (pk, vk) pair per bucket; setup is program-agnostic.

13.8 Verification Rules

1. Groth16 proof verifies against vk for the declared `size_bucket`.
2. `Poseidon(received_ciphertext) == ciphertext_hash`.
3. `schema_hash` matches the program's declared schema.
4. Entry size (post-padding) fits within `size_bucket`.

14 Standard Programs

14.1 ZID (Zero Identity)

Field	Value
`name`	`"zid"`
`version`	`1`
`proof_required`	`false`

ZidMessage: `owner_did` (text), `display_name` (optional text), `timestamp_ms` (uint64), `signature` (PQ-hybrid, §7).

14.2 Interlink

v2 Descriptor (current default):

Field	Value
`name`	`"interlink"`
`version`	`2`
`proof_required`	`true`
`proof_system`	`Groth16`

ZMessage: `sender_did`, `channel_id`, `content` (UTF-8), `timestamp_ms`, `signature`. Maximum encoded message size: 64 KB.

Channel-to-sector mapping:

```
SectorId = SHA-256("interlink/channel/"
                || channel_id_bytes)
```

ZID (v1) and Interlink (v2) are **default programs**: ZODES subscribe to them automatically.

15 Services Layer

The Services layer provides a framework for stateless servers that run on ZODES, use Programs for persistent state, and expose HTTP endpoints. Programs are passive descriptors that define storage schemas; Services are active processes with runtime lifecycle, request handling, and background tasks. A Service never holds authoritative local state—it reads and writes exclusively through Programs, so any ZODE running the same service can serve the same requests without migration or synchronization.

15.1 ServiceDescriptor

A CBOR map with the following fields:

Field	Type	Description
name	text string	Human-readable service name
version	text string	Semantic version
required_programs	list of ProgramId	Programs this service reads/writes (must already exist)
owned_programs	list of ProgramDescriptor	Programs this service defines; auto-registered when the service is enabled

The ServiceId is derived as SHA-256(CBOR(ServiceDescriptor)), mirroring the ProgramId pattern (§4.1). Services can both depend on existing Programs and bring their own storage schemas via owned_programs.

15.2 Service Trait

Every service—native or WASM—implements four operations:

Operation	Description
descriptor()	Returns the service's ServiceDescriptor; declares identity and program dependencies.
routes(ctx)	Returns an HTTP router mounted at /services/{service_id_hex}/.
on_start(ctx)	Called after ZODE boot; may spawn background tasks using a cancellation token for graceful shutdown.
on_stop()	Cleanup hook called during ZODE shutdown.

15.3 ServiceContext and ProgramStore

The ServiceContext is a service's interface to the ZODE. It provides:

- **ProgramStore**—a key-value abstraction over sector append-logs. Keys map to SectorId via SHA-256(key). put appends a new entry (latest entry = current value). get reads the last entry. list reads the full log history.
- **Ephemeral tokens**—HMAC-SHA256 signed, time-limited tokens for stateless short-lived flows (auth challenges, OAuth nonces). The client holds the token and presents it back; no server-side storage is required.
- **Event broadcast**—services emit Started, Stopped, and RequestHandled events.
- **Shutdown token**—cooperative cancellation for background tasks.

ProgramStore Operations

Operation	Behavior
get(key)	Derive SectorId = SHA-256(key), read last log entry.
put(key, value)	Derive SectorId, append value to sector log.
list(key)	Read all entries for the derived sector (full history).
list_from(key, from)	Read entries starting from a given index.
len(key)	Return the number of entries for the derived sector.

15.4 ServiceRegistry

The ServiceRegistry manages the lifecycle of all active services on a ZODE:

- **Register**—compute ServiceId, reject duplicates, store the service.
- **Start all**—create a ServiceContext per service, call on_start, emit Started event.
- **Stop all**—cancel background tasks, call on_stop, emit Stopped event.
- **Merged router**—build a combined HTTP router with each service nested at /services/{service_id_hex}/.
- **Required programs**—collect the union of all registered services' required and owned program IDs for topic subscription.

15.5 HTTP Integration

Service HTTP routes are merged into the same server that hosts the JSON-RPC sector endpoint. The resulting route layout is:

```
POST /rpc                    -- JSON-RPC sector operations (§11)
GET|POST /services/{service_id}/... -- per-service endpoints
```

Each service defines its own sub-routes (e.g. /resolve, /health, /messages). The ZODE handles TLS, CORS, and connection management; services only see deserialized HTTP requests.

15.6 WASM Runtime

The Services layer supports a sandboxed WASM runtime for third-party services using the WebAssembly Component Model. A WIT (WebAssembly Interface Types) contract defines the host–guest boundary:

Host-provided imports:

Interface	Functions	Description
store	get, put, list-entries, list-from, entry-count	Key-value access to Grid Programs
ephemeral	create-token, verify-token	Signed time-limited tokens

Guest-exported interface:

Function	Description
get-descriptor()	Returns the service descriptor
handle-request(req)	Processes an HTTP request and returns a response
on-start()	Lifecycle hook at service startup
on-stop()	Lifecycle hook at service shutdown

WASM services are subject to resource limits: CPU fuel metering, memory caps, and per-request wall-clock timeouts. A misbehaving module is terminated without affecting the ZODE or other services.

15.7 Standard Services

Two services are registered by default:

Service	Required Programs	Endpoints
Identity (v1.0.0)	ZID v1	POST /resolve, GET /health
Interlink (v1.0.0)	Interlink v2	GET /messages, GET /health

The Identity service wraps the ZID Program with a DID resolution HTTP API. The Interlink service exposes channel message retrieval over HTTP. Both are native (compiled) services and are always registered on ZODE startup.

15.8 Programs vs. Services

Aspect	Program	Service
Nature	Passive descriptor (schema + proof config)	Active process (server, compute)
State	Defines storage format	Stateless; reads/writes via Programs
Identity	`ProgramId = SHA-256(descriptor)`	`ServiceId = SHA-256(descriptor)`
Runs on	Nothing (specification only)	ZODE nodes
Exposed API	None (sector protocol only)	HTTP routes, background tasks

16 ZODE Behavior

16.1 Startup Sequence

1. Open persistent storage.
2. Start libp2p swarm (GossipSub, request-response, Kademlia).
3. Build `ServiceRegistry`; register default services (Identity, Interlink).
4. Compute effective topic set: default programs ∪ configured programs ∪ service-required programs.
5. Subscribe to each GossipSub topic.
6. Start all registered services (`on_start`); build merged HTTP router.
7. Start RPC server with sector endpoint and service routes merged.
8. Dial bootstrap peers.
9. If Kademlia enabled, seed routing table and bootstrap.
10. Enter event loop.

16.2 Event Loop

The ZODE continuously processes: incoming sector requests, gossip messages, peer events, Kademlia discovery, publish queue, service events, and shutdown signals.

16.3 Append + Gossip Flow

On Append: (1) validate program, sector filter, entry size, shape proof; (2) append to local storage; (3) send success response. Gossip propagation is **client-triggered**: after successful append, the client publishes `GossipSectorAppend` to the program topic.

16.4 Shutdown Sequence

On shutdown: (1) cancel service shutdown tokens; (2) call `on_stop` on each registered service; (3) stop the RPC server; (4) close the libp2p swarm; (5) flush and close persistent storage.

17 Visibility and Privacy Properties

What a ZODE can see

Information	Visible?
`program_id`	Yes — routing and policy
`sector_id`	Yes — opaque 32 bytes
Payload size	Yes — mitigated by padding
Entry timing	Yes — append/read arrival
Batched sector IDs	Yes — within one request
Client IP	Yes — transport-level
Service requests	Yes — ZODE routes HTTP to services

What a ZODE cannot see

Information	Why
Entry content/structure	Encrypted (XChaCha20/Poseidon)
Author identity	Inside encrypted payload
Access control	Key material never on wire
Sector relationships	HKDF-derived IDs are unlinkable
Ordering/timestamps	Inside encrypted payload

18 Security Considerations

- **No transport anonymity.** IP addresses are visible. Onion routing is out of scope.
- **Timing correlation.** A ZODE can correlate writes/reads from the same connection.
- **No wire-level write authorization.** Any client with a ProgramId can append. Access control is application-layer.
- **Gossip propagation delay.** Entries initially exist on one ZODE. Clients MAY multi-send.
- **Key compromise.** NeuralKey compromise implies all derived keys are compromised. Shamir splitting mitigates single-point-of-failure.
- **Post-quantum readiness.** Hybrid constructions require breaking both classical and PQ components.
- **WASM service isolation.** Sandboxed WASM services are subject to fuel metering, memory limits, and wall-clock timeouts. A misbehaving module cannot affect the ZODE or other services, but resource exhaustion within limits is possible.
- **Service statefulness.** Services are stateless by construction; all persistent state flows through Programs. A compromised service cannot corrupt state beyond what its declared Programs allow.

19 Interoperability Requirements

A conforming Grid implementation MUST:

1. Serialize all protocol messages as deterministic CBOR (RFC 8949 §4.2.1).

2. Derive ProgramIds as `SHA-256(CBOR(ProgramDescriptor))`.
3. Use `/grid/sector/1.0.0` for sector request-response.
4. Use `/grid/kad/1.0.0` for Kademlia DHT.
5. Format GossipSub topics as `prog/<64_hex_chars>`.
6. Serialize `GossipSectorAppend` as CBOR.
7. Use XChaCha20-Poly1305 or Poseidon sponge per ProgramDescriptor.
8. Implement the padding bucket scheme (§6.4).
9. Use HKDF-SHA256 with exact domain separation strings (§5.2).
10. Produce hybrid signatures: Ed25519 (64 B) + ML-DSA-65 (3,309 B); both must verify.
11. Perform hybrid key encapsulation: X25519 + ML-KEM-768 via HKDF (§5.5).
12. Reject non-32-byte sector IDs.
13. Enforce batch limits: 64 entries, 4 MB total.
14. Accept gossip with conflict resolution per §12.2.
15. Derive ServiceIds as `SHA-256(CBOR(ServiceDescriptor))`.
16. Mount service HTTP routes at `/services/{service_id_hex}/` alongside the JSON-RPC endpoint.
17. Format service GossipSub topics as `svc/<64_hex_chars>`.
18. Map ProgramStore keys to SectorIds via `SHA-256(key)`; `put` appends, `get` reads last entry.